

Practical 0

- Python Installation



Practical 1

Aim: Set up Python using a virtual environment, create basic variables, and demonstrate different data types (int, float, string, list, tuple, dictionary, set).

Source-Code:

```
# 1. Integer (int)
age = 22
print("Age:", age)
print("Type of age:", type(age))

# 2. Float
height = 5.11
print("Height:", height)
print("Type of height:", type(height))

print("-----")

# 3. String
name = "NotGonnaTell"
print("Name:", name)
print("Type of name:", type(name))

print("-----")

# 4. List (Ordered, Changeable)
fruits = ["apple", "banana", "mango"]
print("Fruits List:", fruits)
print("Type of fruits:", type(fruits))

print("-----")

# 5. Tuple (Ordered, Not Changeable)
coordinates = (10, 20)
print("Coordinates Tuple:", coordinates)
print("Type of coordinates:", type(coordinates))

print("-----")

# 6. Dictionary (Key-Value Pair)
student = {
    "name": "NotGonnaTell",
    "age": 22,
    "course": "MCA"
}
print("Student Dictionary:", student)
print("Type of student:", type(student))

print("-----")

# 7. Set (Unordered, Unique values)
numbers = {1, 2, 3, 3, 4}
print("Set of numbers:", numbers)
print("Type of numbers:", type(numbers))
```

Output:

Age: 22

Type of age: <class 'int'>

Height: 5.11

Type of height: <class 'float'>

Name: NotGonnaTell

Type of name: <class 'str'>

Fruits List: ['apple', 'banana', 'mango']

Type of fruits: <class 'list'>

Coordinates Tuple: (10, 20)

Type of coordinates: <class 'tuple'>

Student Dictionary: {'name': 'NotGonnaTell', 'age': 22, 'course': 'MCA'}

Type of student: <class 'dict'>

Set of numbers: {1, 2, 3, 4}

Type of numbers: <class 'set'>

Practical 2

Aim: Write a script that performs arithmetic operations, string manipulations (concatenation, slicing), and type conversions.

Source-Code:

```
# 1. Arithmetic Operations
# -----

a = 10
b = 5

print("Arithmetic Operations:")
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Modulus (Remainder):", a % b)

print("\n-----")
# -----
# 2. String Manipulation
# -----

str1 = "Hello"
str2 = "World"

# Concatenation (joining strings)
result = str1 + " " + str2
print("String Concatenation:", result)

# Slicing (extracting part of string)
print("First 5 characters:", result[0:5])
print("Last 5 characters:", result[6:11])

print("\n-----")
# -----
# 3. Type Conversion
# -----

num_str = "100"

# Convert string to integer
num_int = int(num_str)
print("String to Integer:", num_int)

# Convert integer to float
num_float = float(num_int)
print("Integer to Float:", num_float)

# Convert number to string
num_to_str = str(num_float)
print("Float to String:", num_to_str)
))
```

Output:

Arithmetic Operations:

Addition: 15

Subtraction: 5

Multiplication: 50

Division: 2.0

Modulus (Remainder): 0

String Concatenation: Hello World

First 5 characters: Hello

Last 5 characters: World

String to Integer: 100

Integer to Float: 100.0

Float to String: 100.0

Practical 3

Aim: Import a dataset with missing values, inspect the data, and handle missing values using techniques like deletion, mean imputation, and forward/backward fill.

Source-Code:

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Aman", "Riya", "Rahul", "Sneha", "Karan"],
    "Age": [20, 21, np.nan, 22, 23],
    "Marks": [85, np.nan, 78, 90, np.nan]
}
df = pd.DataFrame(data)

print("===== Original Dataset =====\n")
print(df)

print("\n===== Dataset Information =====\n")
print(df.info())

print("\n===== Missing Values =====\n")
print(df.isnull().sum())

deleted_data = df.dropna()

print("\n===== Dataset After Removing Missing Values =====\n")
print(deleted_data)

mean_data = df.copy()

age_mean = mean_data["Age"].mean()

mean_data["Age"] = mean_data["Age"].fillna(age_mean)

marks_mean = mean_data["Marks"].mean()

mean_data["Marks"] = mean_data["Marks"].fillna(marks_mean)

print("\n===== Dataset After Mean Imputation =====\n")
print(mean_data)

forward_fill_data = df.ffill()

print("\n===== Dataset After Forward Fill =====\n")
print(forward_fill_data)

backward_fill_data = df.bfill()

print("\n===== Dataset After Backward Fill =====\n")
print(backward_fill_data)
```

Output:

===== Original Dataset =====

	Name	Age	Marks
0	Aman	20.0	85.0
1	Riya	21.0	NaN
2	Rahul	NaN	78.0
3	Sneha	22.0	90.0
4	Karan	23.0	NaN

===== Dataset Information =====

```
<class 'pandas.DataFrame'>  
RangeIndex: 5 entries, 0 to 4  
Data columns (total 3 columns):  
#   Column  Non-Null Count  Dtype  
---  ---  
0   Name    5 non-null       str  
1   Age     4 non-null       float64  
2   Marks   3 non-null       float64  
dtypes: float64(2), str(1)  
memory usage: 275.0 bytes  
None
```

===== Missing Values =====

```
Name      0  
Age       1  
Marks     2  
dtype: int64
```

===== Dataset After Removing Missing Values =====

	Name	Age	Marks
0	Aman	20.0	85.0
3	Sneha	22.0	90.0

===== Dataset After Mean Imputation =====

	Name	Age	Marks
0	Aman	20.0	85.000000
1	Riya	21.0	84.333333
2	Rahul	21.5	78.000000
3	Sneha	22.0	90.000000
4	Karan	23.0	84.333333

===== Dataset After Forward Fill =====

	Name	Age	Marks
0	Aman	20.0	85.0
1	Riya	21.0	85.0
2	Rahul	21.0	78.0
3	Sneha	22.0	90.0
4	Karan	23.0	90.0

===== Dataset After Backward Fill =====

	Name	Age	Marks
0	Aman	20.0	85.0
1	Riya	21.0	78.0
2	Rahul	22.0	78.0
3	Sneha	22.0	90.0
4	Karan	23.0	NaN

Practical 4

Aim: Use Python libraries like Pandas and Numpy to identify outliers using statistical methods (IQR, Z-score) and handle them by removing or capping.

Source-Code:

```
import pandas as pd
import numpy as np

data = {'Marks': [10, 12, 14, 15, 18, 20, 22, 25, 100]}
df = pd.DataFrame(data)
print("Original Data:")
print(df)

Q1 = df['Marks'].quantile(0.25)
Q3 = df['Marks'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
print("\nLower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
outliers = df[(df['Marks'] < lower_limit) |
               (df['Marks'] > upper_limit)]
print("\nOutliers:")
print(outliers)
print("\nCleaned Data:")
clean_data = df[(df['Marks'] >= lower_limit) &
                 (df['Marks'] <= upper_limit)]
print(clean_data)
```

Output:

Original Data:

	Marks
0	10
1	12
2	14
3	15
4	18
5	20
6	22
7	25
8	100

Lower Limit: 2.0

Upper Limit: 34.0

Outliers:

	Marks
8	100

Cleaned Data:

	Marks
0	10
1	12
2	14
3	15
4	18
5	20
6	22
7	25

Practical 5

Aim: Demonstrate normalization, standardization, and data encoding on a sample dataset using Python.

Source-Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
data = {
    'Name': ['Amit', 'Priya', 'Rahul', 'Sneha', 'Karan'],
    'Age': [22, 25, 28, 24, 30],
    'Salary': [25000, 30000, 35000, 28000, 40000],
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR']
}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
print("\n----- Normalization -----")
normalizer = MinMaxScaler()
df[['Age_Normalized', 'Salary_Normalized']] = normalizer.fit_transform(
    df[['Age', 'Salary']]
)
print(df[['Age', 'Age_Normalized',
           'Salary', 'Salary_Normalized']])
print("\n----- Standardization -----")
standardizer = StandardScaler()
df[['Age_Standardized', 'Salary_Standardized']] = standardizer.fit_transform(
    df[['Age', 'Salary']]
)
print(df[['Age', 'Age_Standardized',
           'Salary', 'Salary_Standardized']])

print("\n----- Label Encoding -----")
label_encoder = LabelEncoder()
df['Gender_Encoded'] = label_encoder.fit_transform(df['Gender'])
print(df[['Gender', 'Gender_Encoded']])
print("\n----- One-Hot Encoding -----")
department_encoded = pd.get_dummies(df['Department'])
print(department_encoded)
print("\n----- Final Dataset -----")
print(df)
```

Output:

Original Dataset:

	Name	Age	Salary	Gender	Department
0	Amit	22	25000	Male	HR
1	Priya	25	30000	Female	IT
2	Rahul	28	35000	Male	Finance
3	Sneha	24	28000	Female	IT
4	Karan	30	40000	Male	HR

----- Normalization -----

	Age	Age_Normalized	Salary	Salary_Normalized
0	22	0.000	25000	0.000000
1	25	0.375	30000	0.333333
2	28	0.750	35000	0.666667
3	24	0.250	28000	0.200000
4	30	1.000	40000	1.000000

----- Standardization -----

	Age	Age_Standardized	Salary	Salary_Standardized
0	22	-1.330266	25000	-1.241971
1	25	-0.280056	30000	-0.301084
2	28	0.770154	35000	0.639803
3	24	-0.630126	28000	-0.677439
4	30	1.470294	40000	1.580691

----- Label Encoding -----

	Gender	Gender_Encoded
0	Male	1
1	Female	0
2	Male	1
3	Female	0
4	Male	1

----- One-Hot Encoding -----

	Finance	HR	IT
0	False	True	False
1	False	False	True
2	True	False	False
3	False	False	True
4	False	True	False

----- Final Dataset -----

	Name	Age	Salary	Gender	Department	Age_Normalized	Salary_Normalized	Age_Standardized	Salary_Standardized	Gender_Encoded
0	Amit	22	25000	Male	HR	0.000	0.000000	-1.330266	-1.241971	1
1	Priya	25	30000	Female	IT	0.375	0.333333	-0.280056	-0.301084	0
2	Rahul	28	35000	Male	Finance	0.750	0.666667	0.770154	0.639803	1
3	Sneha	24	28000	Female	IT	0.250	0.200000	-0.630126	-0.677439	0
4	Karan	30	40000	Male	HR	1.000	1.000000	1.470294	1.580691	1

Practical 6

Aim: Create Numpy arrays, perform basic operations (addition, multiplication), and apply statistical functions (mean, median, standard deviation).

Source-Code:

```
import numpy as np
arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([1, 2, 3, 4, 5])
print("First Array:")
print(arr1)
print("\nSecond Array:")
print(arr2)
addition = arr1 + arr2
print("\nAddition of Arrays:")
print(addition)
multiplication = arr1 * arr2
print("\nMultiplication of Arrays:")
print(multiplication)
mean_value = np.mean(arr1)
print("\nMean:")
print(mean_value)
median_value = np.median(arr1)
print("\nMedian:")
print(median_value)
std_value = np.std(arr1)
print("\nStandard Deviation:")
print(std_value)
```

Output:

First Array:
[10 20 30 40 50]

Second Array:
[1 2 3 4 5]

Addition of Arrays:
[11 22 33 44 55]

Multiplication of Arrays:
[10 40 90 160 250]

Mean:
30.0

Median:
30.0

Standard Deviation:
14.142135623730951